

Big Data - TG CPGET 2026 Revision Handbook

BIG DATA - TG CPGET 2026 Revision Handbook

A concept-to-practice guide covering: Overview of Big Data, Technologies for Handling Big Data, Hadoop Ecosystem, HDFS, HBase, MapReduce Fundamentals, Big Data Technology Foundations, Databases & Data Warehouses, NoSQL Data Management.

HOW TO USE THIS HANDBOOK

Each major block has: **Concept in Simple Language -> Key Definitions/Formulas -> Examples -> Practice Questions**. Read once for understanding, then use the “Key Definitions” boxes as flash-cards before the exam.

BLOCK 1: OVERVIEW OF BIG DATA & TECHNOLOGIES FOR HANDLING BIG DATA

1.1 What is Big Data?

Big Data means datasets so large, fast-moving, or messy that ordinary software (like Excel or a single database server) cannot store, process, or analyze them efficiently. It's not just about “a lot of data” - it's about data that breaks traditional tools.

1.2 The Characteristics (V's) of Big Data

V	Meaning	Simple Explanation
Volume	Size of data	Terabytes to petabytes - e.g., Facebook's daily uploads
Velocity	Speed of generation/processing	Stock market ticks, sensor streams arriving every second
Variety	Different data types	Structured (tables), semi-structured (XML/JSON), unstructured (video, text) Noisy, incomplete, or

Veracity	Trustworthiness/quality	inconsistent data
Value	Usefulness	Data is worthless unless it generates insight/profit
<i>(Sometimes added)</i> Variability	Inconsistent meaning over time	Same word means different things in different contexts

Exam Tip: The “3 core V’s” (Volume, Velocity, Variety) were coined by Doug Laney (Gartner, 2001). Later Veracity and Value were added - the “5 V’s” model is a favorite MCQ source.

1.3 Structured vs Semi-Structured vs Unstructured Data

- **Structured:** Fixed schema, fits neatly in rows/columns (RDBMS tables, Excel sheets).
- **Semi-structured:** Has some organizational tags but no rigid schema (JSON, XML, email headers).
- **Unstructured:** No predefined format (images, video, audio, free text, social media posts). ~80% of enterprise data is unstructured.

1.4 Why Traditional Systems Fail for Big Data

Traditional RDBMS scale **vertically** (adding more power to one machine - CPU/RAM/disk), which is expensive and has a hard ceiling. Big Data systems scale **horizontally** (adding more commodity machines to a cluster), which is cheap and nearly limitless.

Aspect	Traditional RDBMS	Big Data Systems
Scaling	Vertical (scale-up)	Horizontal (scale-out)
Schema	Fixed, defined upfront (schema-on-write)	Flexible, applied at read time (schema-on-read)
Hardware	Expensive, specialized	Cheap, commodity
Data type	Structured only	All types
Fault tolerance	Limited (single point of failure)	Built-in (replication across nodes)

1.5 Technologies for Handling Big Data (Landscape)

- **Storage:** HDFS, Amazon S3, Google Cloud Storage
- **Processing (Batch):** MapReduce, Apache Spark, Apache Pig, Apache Hive
- **Processing (Real-time/Streaming):** Apache Storm, Apache Spark Streaming, Apache Kafka, Apache Flink
- **NoSQL Databases:** HBase, MongoDB, Cassandra, Redis, Neo4j
- **Resource Management:** YARN (Yet Another Resource Negotiator)
- **Workflow/Coordination:** Apache Oozie, Apache ZooKeeper
- **Data Ingestion:** Apache Sqoop (RDBMS <-> Hadoop), Apache

Flume (log/streaming data)

- **Query Layer:** Apache Hive (SQL-like), Apache Pig (scripting)

Key Definitions Box

- **Scalability:** The ability of a system to handle growing amounts of data by adding resources.
- **Fault tolerance:** The system continues to work correctly even if a node/component fails.
- **Commodity hardware:** Inexpensive, standard computers (not specialized supercomputers) used in clusters.
- **Data lake:** A repository storing raw data in native format (structured + unstructured) until needed.

1.6 Illustrative Examples

1. **Volume:** YouTube uploads ~500 hours of video every minute - a single server cannot store or index this.
2. **Velocity:** Credit card fraud detection systems must analyze transactions within milliseconds as they occur.
3. **Variety:** A hospital's data includes structured patient records (tables), semi-structured lab reports (XML), and unstructured MRI scans (images).
4. **Veracity:** Twitter/X sentiment data contains sarcasm, typos, and bots - making trustworthiness a challenge.
5. **Horizontal scaling:** A Hadoop cluster of 100 cheap machines outperforms one expensive supercomputer for storing 1 petabyte of log data, and if 2 machines fail, the cluster still works.

1.7 Practice Questions - Block 1

1. Which of the following is **not** one of the original 3 V's of Big Data proposed by Gartner?
a. Volume (b) Velocity (c) Variety (d) Veracity
2. Scaling a system by adding more machines to a cluster (rather than upgrading a single machine) is called:
a. Vertical scaling (b) Horizontal scaling (c) Diagonal scaling (d) Elastic scaling
3. Data stored in JSON or XML format, which has some structure but no fixed schema, is classified as:
a. Structured (b) Semi-structured (c) Unstructured (d) Relational
4. The "Veracity" characteristic of Big Data primarily refers to:
a. Speed of data generation (b) Size of dataset (c) Trustworthiness/quality of data (d) Type of data
5. Which tool is primarily used to ingest data from relational databases into Hadoop?
a. Flume (b) Sqoop (c) Oozie (d) ZooKeeper
6. A repository that stores raw, unprocessed data of all types until it is needed for analysis is known as a:
a. Data warehouse (b) Data mart (c) Data lake (d) Data cube
7. Approximately what percentage of enterprise data is estimated to be unstructured?
a. 20% (b) 50% (c) 80% (d) 95%
8. Which characteristic of Big Data deals with data whose meaning or format changes over time/context?
a. Velocity (b) Variability (c) Value (d) Veracity
9. Traditional RDBMS enforce schema at the time of writing data.

This approach is called:

- a. Schema-on-read (b) Schema-on-write (c) Schema-free (d) Dynamic schema

10. Which of these is a tool used for ingesting streaming/log data (e.g., server logs) into Hadoop?
- a. Sqoop (b) Flume (c) Hive (d) Pig
-

BLOCK 2: HADOOP ECOSYSTEM & HDFS

2.1 What is Hadoop?

Hadoop is an open-source framework (Apache project) that allows distributed storage and processing of huge datasets across clusters of commodity computers using simple programming models. It was inspired by Google's GFS and MapReduce papers, created by Doug Cutting and Mike Cafarella, and named after Cutting's son's toy elephant.

2.2 Core Components of Hadoop (The "Core 4")

1. **HDFS (Hadoop Distributed File System)** - storage layer
2. **MapReduce** - processing/computation layer
3. **YARN** - resource management layer (job scheduling, cluster resource allocation)
4. **Hadoop Common** - shared libraries/utilities used by other modules

2.3 The Hadoop Ecosystem (Beyond the Core)

Tool	Purpose	Analogy
HBase	NoSQL column-oriented database on top of HDFS	"Table on top of files"
Hive	SQL-like querying (HiveQL) over Hadoop data	"SQL for Hadoop"
Pig	Scripting language (Pig Latin) for data transformation	"ETL scripting"
Sqoop	Transfers data between RDBMS and HDFS	"Bridge to relational DBs"
Flume	Collects/aggregates streaming/log data into HDFS	"Log collector"
Oozie	Workflow scheduler for Hadoop jobs	"Job orchestrator"

ZooKeeper	Coordination service (locks, configs, naming) for distributed apps	“Traffic controller”
Mahout	Machine learning library	“ML on Hadoop”
Spark	Fast, in-memory cluster computing (can run on Hadoop/YARN)	“Faster MapReduce”

2.4 HDFS - Hadoop Distributed File System

HDFS stores massive files by splitting them into **blocks** and distributing/replicating those blocks across many machines (nodes), so data survives hardware failure and can be processed in parallel.

Architecture (Master-Slave Model)

- **NameNode** (Master): Stores **metadata** only - filenames, block locations, permissions, directory tree. It does NOT store actual data. There is (in classic Hadoop 1.x) a **Single Point of Failure** risk if NameNode goes down - mitigated in Hadoop 2.x+ with **High Availability (HA)** using a Standby NameNode.
- **DataNode** (Slave): Stores the **actual data blocks**. Reports block status to NameNode via **heartbeats** (every 3 seconds by default).
- **Secondary NameNode**: NOT a backup/failover node (common misconception!). It periodically merges the NameNode’s edit logs with the fsimage (checkpointing) to prevent the edit log from growing too large.

Key Definitions & Formulas Box

- **Block size (default)**: 128 MB in Hadoop 2.x/3.x (was 64 MB in Hadoop 1.x).
- **Replication factor (default)**: 3 - each block is copied to 3 different DataNodes for fault tolerance.
- **Rack Awareness**: HDFS places replicas across different racks (e.g., 2 replicas on one rack, 1 on another) to survive rack-level failures while minimizing network traffic.
- **Heartbeat**: Signal sent by DataNode to NameNode confirming it is alive (default every 3 sec).
- **Fsimage**: A persistent checkpoint/snapshot of the HDFS file system metadata.
- **Edit log**: Records every change made to the file system metadata since the last fsimage.
- **Write-Once-Read-Many (WORM)**: HDFS’s data access model - files, once written, are rarely modified but read many times.

Formula-style Calculation

Total storage consumed by a file = File size x Replication factor
 Example: A 1 GB file with replication factor 3 consumes **3 GB** of physical storage across the cluster.

Number of blocks for a file = $\text{ceil}(\text{File size} / \text{Block size})$ Example:
A 300 MB file with 128 MB block size $\rightarrow \text{ceil}(300/128) = 3$
blocks (128 MB + 128 MB + 44 MB).

2.5 Illustrative Examples

1. **Metadata vs Data:** NameNode is like a library's catalog card system (knows *where* each book is); DataNodes are the actual shelves holding books.
2. **Replication:** A 500 MB log file with default replication (3) will occupy 1500 MB of cluster-wide disk space, spread across 3 different machines.
3. **Block splitting:** A 1 GB video file with 128 MB blocks is split into 8 blocks (7x128MB + 1x104MB), each replicated 3 times = 24 block copies across the cluster.
4. **Rack awareness in action:** If one entire rack loses power, HDFS still serves the file because at least one replica exists on a different rack.
5. **Secondary NameNode role:** Every hour (configurable), it fetches the edit log from NameNode, merges it into fsimage, and sends the updated fsimage back - reducing NameNode restart time.

2.6 Practice Questions - Block 2

1. Which Hadoop component is responsible for storing metadata such as file-to-block mapping?
a. DataNode (b) NameNode (c) Secondary NameNode (d) JobTracker
 2. The default block size in Hadoop 2.x is:
a. 32 MB (b) 64 MB (c) 128 MB (d) 256 MB
 3. The default replication factor in HDFS is:
a. 1 (b) 2 (c) 3 (d) 5
 4. The Secondary NameNode's primary function is to:
a. Act as a hot backup for the NameNode (b) Store actual data blocks (c) Merge edit logs with fsimage periodically (d) Schedule MapReduce jobs
 5. A signal periodically sent by a DataNode to the NameNode to indicate it is functioning is called a:
a. Ping (b) Heartbeat (c) Token (d) Trigger
 6. If a 640 MB file is stored in HDFS with a block size of 128 MB, how many blocks will it be split into?
a. 3 (b) 4 (c) 5 (d) 6
 7. Which tool in the Hadoop ecosystem provides a SQL-like interface for querying data stored in HDFS?
a. Pig (b) Hive (c) Sqoop (d) Flume
 8. HDFS's strategy of placing replicas across different racks to balance fault tolerance and network efficiency is called:
a. Load balancing (b) Rack awareness (c) Data locality (d) Block caching
 9. Which of the following is TRUE about HDFS's data access pattern?
a. Frequent random writes, rare reads (b) Write-once, read-many (c) Read-once, write-many (d) Continuous overwrite
 10. A 1 GB file stored with a replication factor of 3 will consume how much total cluster storage?
a. 1 GB (b) 2 GB (c) 3 GB (d) 4 GB
-

BLOCK 3: MAPREDUCE FUNDAMENTALS & HBase

3.1 MapReduce - The Processing Model

MapReduce is a programming model for processing large datasets in **parallel** across a cluster in two phases: **Map** and **Reduce**. Instead of moving data to the program, MapReduce moves the *program* to where the data lives (**data locality**), which is faster for huge datasets.

The Phases

1. **Input Splitting:** Input data is divided into fixed-size splits.
2. **Map Phase:** Each split is processed by a Mapper, which transforms input into **key-value pairs**.
3. **Shuffle & Sort:** Intermediate key-value pairs are grouped by key and sorted, then sent to reducers.
4. **Reduce Phase:** Reducer aggregates/summarizes values for each key into a final output.
5. **Output:** Final key-value pairs written back to HDFS.

Key Definitions Box

- **Mapper:** Processes input splits and emits intermediate (key, value) pairs.
- **Reducer:** Aggregates all values associated with the same key.
- **Combiner:** A “mini-reducer” run on the Mapper’s output *locally* (on the same node) before shuffling, to reduce network traffic. Optional optimization.
- **Partitioner:** Decides which Reducer a given key-value pair goes to (default: hash of key mod number of reducers).
- **JobTracker / ResourceManager:** In Hadoop 1.x, JobTracker manages jobs; in Hadoop 2.x+ (YARN), **ResourceManager** + **ApplicationMaster** take over this role.
- **TaskTracker / NodeManager:** Executes individual map/reduce tasks on a node.
- **Data Locality:** Running computation on the node where the data already resides, to minimize network transfer.

The Classic Example: Word Count

Input: "big data is big"
Map Output: (big,1) (data,1) (is,1) (big,1)
Shuffle/Sort: big -> [1,1] | data -> [1] | is -> [1]
Reduce Output: (big,2) (data,1) (is,1)

3.2 HBase - NoSQL Database on Hadoop

HBase is an open-source, **column-oriented**, distributed NoSQL database that runs **on top of HDFS**, modeled after Google’s **Bigtable**. It’s designed for real-time random read/write access to huge tables (billions of rows x millions of columns) - something plain HDFS (write-once) cannot do.

HBase Data Model

- **Table:** Collection of rows, sparse and distributed.
- **Row Key:** Unique identifier for a row; rows are sorted lexicographically by row key (this is critical for fast lookups).
- **Column Family:** A grouping of related columns, physically stored together (must be defined at table creation).
- **Column Qualifier:** Specific column name within a column family (can be added dynamically - flexible schema).
- **Cell:** Intersection of row + column family + qualifier, storing a **versioned** value (timestamped).
- **Timestamp:** Each cell value is versioned; HBase keeps a configurable number of versions.

HBase Architecture

- **HMaster:** Manages region assignment, load balancing, schema changes (like a coordinator; does NOT serve data directly).
- **RegionServer:** Serves read/write requests; hosts **Regions** (horizontal partitions of a table by row key range).
- **Region:** A contiguous range of rows; a table is split into regions as it grows.
- **ZooKeeper:** Coordinates HMaster and RegionServers, tracks which server is alive.

HBase vs RDBMS

Feature	RDBMS	HBase
Schema	Fixed	Flexible (columns added dynamically)
Scaling	Vertical	Horizontal
Query language	SQL	Java API / HBase shell (no native SQL)
Joins	Supported	Not supported natively
Best for	Structured, transactional data	Sparse, huge, semi-structured data with fast random access

3.3 Illustrative Examples

1. **MapReduce for log analysis:** Counting how many times each HTTP error code appears in a 10 TB web server log - Mappers emit (error_code, 1), Reducers sum them per code.
2. **Combiner optimization:** In word count over 1000 files, a Combiner pre-aggregates counts on each Mapper node before shuffling, drastically cutting network traffic between Map and Reduce phases.
3. **Data locality:** A MapReduce job processing a file's blocks preferentially runs each Map task on the DataNode that already physically stores that block.
4. **HBase for sensor data:** A telecom company stores billions of call-detail records (CDRs) with row key = phone_number + timestamp,

enabling fast lookup of a subscriber's recent calls.

5. **Column family design:** A social media app's HBase table has column families profile (name, bio) and activity (posts, likes) - stored separately for efficient access based on use case.

3.4 Practice Questions - Block 3

1. In MapReduce, the phase where intermediate key-value pairs are grouped by key before reaching the Reducer is called:
a. Splitting (b) Shuffle and Sort (c) Partitioning only (d) Combining only
 2. A "mini-reducer" that runs on the Mapper's local output to reduce data sent over the network is called a:
a. Partitioner (b) Combiner (c) Splitter (d) Aggregator
 3. In Hadoop 2.x with YARN, which component replaced the classic JobTracker's resource management role?
a. TaskTracker (b) ApplicationMaster/ResourceManager (c) NameNode (d) HMaster
 4. HBase's data model is best described as:
a. Row-oriented relational (b) Column-oriented NoSQL (c) Graph-based (d) Document-based
 5. In HBase, rows in a table are physically sorted by:
a. Timestamp (b) Column family name (c) Row key (d) Cell value
 6. Which component in HBase architecture is responsible for region assignment and load balancing (but not serving data directly)?
a. RegionServer (b) HMaster (c) ZooKeeper (d) NameNode
 7. HBase was primarily modeled after which Google technology?
a. GFS (b) Bigtable (c) MapReduce (d) Spanner
 8. In MapReduce's "data locality" principle, computation is preferably moved:
a. To a central server (b) To the node where data resides (c) To the client machine (d) To the NameNode
 9. A horizontal partition of an HBase table, based on a contiguous row-key range, is called a:
a. Shard (b) Region (c) Block (d) Partition
 10. Which of the following is NOT true of HBase compared to a traditional RDBMS?
a. HBase supports flexible schema (b) HBase natively supports complex SQL joins (c) HBase scales horizontally (d) HBase is good for sparse data
-

BLOCK 4: BIG DATA TECHNOLOGY FOUNDATIONS & STORING DATA IN DATABASES/WAREHOUSES

4.1 Technology Foundations - The Building Blocks

Big Data solutions are typically built as a **layered architecture**: 1. **Data Sources Layer:** Sensors, logs, social media, transactional systems. 2. **Ingestion Layer:** Sqoop, Flume, Kafka - bring data in

(batch or streaming). 3. **Storage Layer**: HDFS, NoSQL DBs, cloud object storage. 4. **Processing Layer**: MapReduce, Spark, Hive, Pig. 5. **Analytics/Visualization Layer**: BI tools, dashboards, ML models.

Distributed Computing Principles Underlying Big Data

- **CAP Theorem**: A distributed system can guarantee only 2 of 3: Consistency, Availability, Partition tolerance. Since network partitions are unavoidable in distributed systems, real systems trade off Consistency vs Availability (P is mandatory).
- **BASE vs ACID**:
 - **ACID** (traditional RDBMS): Atomicity, Consistency, Isolation, Durability - strict, guarantees correctness.
 - **BASE** (NoSQL/Big Data systems): Basically Available, Soft state, Eventually consistent - prioritizes availability/speed over immediate consistency.
- **Sharding**: Splitting a large dataset horizontally across multiple servers.
- **Replication**: Copying data across nodes for fault tolerance and availability.

4.2 Databases vs Data Warehouses vs Data Lakes

Feature	Operational Database (OLTP)	Data Warehouse (OLAP)	Data Lake
Purpose	Day-to-day transactions	Historical analysis/reporting	Store raw data for future/flexible use
Data	Current, structured	Historical, structured, cleaned	Raw, all formats
Schema	Schema-on-write	Schema-on-write	Schema-on-read
Users	Clerks, applications	Analysts, managers	Data scientists
Example	Bank's transaction system	Company's sales-trend reporting system	Hadoop cluster storing all raw logs

4.3 Data Warehousing Concepts

- **ETL (Extract, Transform, Load)**: Extract data from sources -> transform/clean it -> load into the warehouse. (ELT reverses the last two steps - common in modern cloud/Big Data pipelines: load raw data first, transform later.)
- **Star Schema**: A central **fact table** (measurable events, e.g., sales) connected to multiple **dimension tables** (descriptive attributes, e.g., time, product, customer). Simple, denormalized, fast for queries.
- **Snowflake Schema**: A star schema where dimension tables are

further normalized into sub-dimensions - saves storage but adds join complexity.

- **Fact Table:** Contains quantitative data (measures) and foreign keys to dimension tables.
- **Dimension Table:** Contains descriptive/categorical attributes used to filter/group facts.
- **OLAP (Online Analytical Processing):** Multidimensional analysis of data (slice, dice, drill-down, roll-up) for decision-making.
- **OLTP (Online Transaction Processing):** Fast, day-to-day transactional operations (insert/update/delete).

4.4 Illustrative Examples

1. **CAP Theorem in practice:** During a network partition, an e-commerce cart system might choose **Availability** (let users keep shopping, sync later) over strict Consistency - this is an "AP" system.
2. **ACID example:** A bank transfer must be Atomic - either both the debit and credit happen, or neither does (no partial transfer).
3. **BASE example:** A social media "like" counter may show slightly outdated counts briefly across servers before "eventually" converging to the correct number.
4. **Star schema:** A retail sales fact table (with measures: quantity, revenue) connects to dimension tables Time, Store, Product, Customer.
5. **ETL vs ELT:** A traditional bank warehouse cleans and transforms data before loading (ETL); a Big Data pipeline loads raw JSON logs into a data lake first, transforming only when needed for a specific analysis (ELT).

4.5 Practice Questions - Block 4

1. According to the CAP theorem, a distributed system can simultaneously guarantee at most:
 - a. All 3 of Consistency, Availability, Partition tolerance
 - (b) Any 2 of the 3
 - (c) Only Consistency
 - (d) Only Partition tolerance
2. The NoSQL consistency model that favors availability and eventual correctness over strict immediate consistency is called:
 - a. ACID
 - (b) BASE
 - (c) CAP
 - (d) SQL
3. In a star schema, the central table containing measurable, quantitative data is called the:
 - a. Dimension table
 - (b) Fact table
 - (c) Lookup table
 - (d) Staging table
4. Which schema type further normalizes dimension tables into sub-dimensions?
 - a. Star schema
 - (b) Snowflake schema
 - (c) Galaxy schema
 - (d) Flat schema
5. OLAP systems are primarily designed for:
 - a. Fast transactional updates
 - (b) Multidimensional analytical queries
 - (c) Real-time locking
 - (d) Data ingestion only
6. "Atomicity" in the ACID model guarantees that:
 - a. Data is eventually consistent
 - (b) A transaction either completes fully or not at all
 - (c) Data is always available
 - (d) Queries run in parallel
7. A repository that stores raw data in its native format, applying schema only when the data is read, follows which approach?

- a. Schema-on-write (b) Schema-on-read (c) Fixed schema (d) No schema
 8. Splitting a large dataset horizontally across multiple servers, each holding a subset of rows, is called:
 - a. Replication (b) Partitioning/Sharding (c) Indexing (d) Normalization
 9. ETL stands for:
 - a. Extract, Transfer, Log (b) Extract, Transform, Load (c) Evaluate, Transform, Load (d) Extract, Test, Load
 10. A traditional day-to-day banking transaction system (handling deposits/withdrawals) is an example of:
 - a. OLAP (b) OLTP (c) Data Lake (d) Data Warehouse
-

BLOCK 5: NoSQL DATA MANAGEMENT

5.1 What is NoSQL?

NoSQL (“Not Only SQL”) databases are non-relational databases designed for flexible schemas, horizontal scalability, and handling large volumes of unstructured/semi-structured data - the opposite design philosophy of rigid, table-based RDBMS.

5.2 The Four Major Types of NoSQL Databases

Type	Data Model	Example DBs	Best Use Case
Key-Value Store	Simple key -> value pairs (like a hash map)	Redis, DynamoDB, Riak	Caching, session storage
Document Store	Semi-structured documents (JSON/BSON)	MongoDB, CouchDB	Content management, catalogs
Column-Family Store	Rows with dynamic columns grouped in families	HBase, Cassandra	Large-scale analytics, time-series
Graph Database	Nodes + edges representing relationships	Neo4j, ArangoDB	Social networks, recommendation engines, fraud detection

5.3 Why NoSQL? (Motivations)

- Handle **massive scale** that RDBMS struggle with (horizontal scaling).
- Handle **unstructured/semi-structured** data without rigid

schema.

- Provide **high availability** and **fast writes** for web-scale applications.
- Trade strict consistency for speed/flexibility (BASE model, per CAP theorem).

5.4 NoSQL vs RDBMS - Quick Comparison

Feature	RDBMS	NoSQL
Schema	Fixed, rigid	Dynamic, flexible
Scaling	Vertical	Horizontal
Consistency model	ACID	BASE (mostly)
Query language	SQL	Varies (API/query language per DB)
Relationships	Strong (foreign keys, joins)	Weak/denormalized (except graph DBs)
Best fit	Transactional, structured data	Big Data, high-scale, flexible-schema apps

5.5 Key Definitions Box

- **Denormalization:** Intentionally duplicating data to avoid expensive joins - common in NoSQL for read performance.
- **Eventual Consistency:** All replicas will converge to the same value *eventually*, but may be temporarily inconsistent.
- **Polyglot Persistence:** Using different types of databases (SQL + multiple NoSQL types) within one application, each for the task it's best suited to.
- **Sharding:** Distributing data across nodes based on a shard key.
- **Document:** A self-contained unit of semi-structured data (e.g., JSON object) in a document store.

5.6 Illustrative Examples

1. **Key-Value:** An e-commerce site stores user shopping-cart sessions in Redis with `session_id` as key and cart contents as value, for millisecond-fast retrieval.
2. **Document store:** MongoDB stores a product catalog where each product document has different attributes (a book has "author"/"pages", a laptop has "RAM"/"processor") - impossible to do cleanly in a single rigid RDBMS table.
3. **Column-family:** Cassandra stores time-series sensor data with row key = `sensor_id`, and columns added dynamically for each new timestamp reading.
4. **Graph database:** Neo4j models a social network where Person nodes connect via `FRIENDS_WITH` edges, making "friends of friends" queries extremely fast compared to SQL joins.
5. **Polyglot persistence:** An online retailer uses MongoDB for product catalogs, Redis for session caching, and Neo4j for its recommendation engine - all within one application.

5.7 Practice Questions - Block 5

1. Which type of NoSQL database stores data as simple pairs, similar to a hash map?
a. Document store (b) Key-value store (c) Graph database (d) Column-family store
2. MongoDB is a popular example of which type of NoSQL database?
a. Key-value store (b) Document store (c) Graph database (d) Column-family store
3. Which NoSQL database type is best suited for modeling social network relationships?
a. Key-value store (b) Document store (c) Graph database (d) Column-family store
4. The practice of using multiple different types of databases within a single application, each suited to a specific task, is known as:
a. Sharding (b) Polyglot persistence (c) Replication (d) Normalization
5. Which of the following is a column-family NoSQL database?
a. Redis (b) Neo4j (c) Cassandra (d) CouchDB
6. NoSQL databases generally follow which consistency model, prioritizing availability over immediate consistency?
a. ACID (b) BASE (c) 2PC (d) MVCC
7. Intentionally duplicating data across a NoSQL database to avoid costly joins is called:
a. Sharding (b) Denormalization (c) Indexing (d) Replication
8. Redis is most commonly used as a:
a. Graph database (b) Document store (c) Key-value store for caching (d) Column-family store
9. In a graph database, relationships between entities are represented as:
a. Foreign keys (b) Edges (c) Columns (d) Joins
10. Which of these is NOT typically a strength of NoSQL databases compared to RDBMS?
a. Horizontal scalability (b) Flexible schema (c) Strong native support for complex multi-table joins (d) High write throughput

QUICK-REVISION FORMULA & FACT SHEET

Item	Value/Fact
Default HDFS block size (Hadoop 2.x/3.x)	128 MB
Default HDFS block size (Hadoop 1.x)	64 MB
Default replication factor	3
Heartbeat interval (DataNode -> NameNode)	~3 seconds
No. of blocks formula	$\text{ceil}(\text{File size} / \text{Block size})$
Total storage formula	File size x Replication factor
CAP Theorem	Pick 2 of: Consistency, Availability, Partition tolerance

ACID stands for	Atomicity, Consistency, Isolation, Durability
BASE stands for	Basically Available, Soft state, Eventually consistent
ETL stands for	Extract, Transform, Load
Hadoop creator	Doug Cutting (named after son's toy elephant)
HBase modeled after	Google Bigtable
HDFS modeled after	Google File System (GFS)
MapReduce modeled after	Google's MapReduce paper
4 NoSQL types	Key-Value, Document, Column-Family, Graph

ANSWER KEYS

Block 1: 1-d, 2-b, 3-b, 4-c, 5-b, 6-c, 7-c, 8-b, 9-b, 10-b **Block 2:** 1-b, 2-c, 3-c, 4-c, 5-b, 6-c, 7-b, 8-b, 9-b, 10-c **Block 3:** 1-b, 2-b, 3-b, 4-b, 5-c, 6-b, 7-b, 8-b, 9-b, 10-b **Block 4:** 1-b, 2-b, 3-b, 4-b, 5-b, 6-b, 7-b, 8-b, 9-b, 10-b **Block 5:** 1-b, 2-b, 3-c, 4-b, 5-c, 6-b, 7-b, 8-c, 9-b, 10-c

FINAL REVISION CHECKLIST

- ☐ Can you list and explain all 5 V's of Big Data with one example each?
- ☐ Can you name the 4 core Hadoop components and their roles?
- ☐ Can you calculate number of blocks and total storage for a given file size?
- ☐ Can you explain the difference between NameNode, DataNode, and Secondary NameNode?
- ☐ Can you walk through the Map -> Shuffle/Sort -> Reduce flow with the Word Count example?
- ☐ Can you draw the HBase architecture (HMaster, RegionServer, Region, ZooKeeper)?
- ☐ Can you state CAP theorem and give an example of an AP vs CP system?
- ☐ Can you differentiate ACID vs BASE with one example each?
- ☐ Can you distinguish Star vs Snowflake schema and Fact vs Dimension tables?
- ☐ Can you name all 4 NoSQL types with one database example and one use case each?